OTIMIZAÇÃO DE MAKESPAN EM PROJETOS DE SOFTWARE: ALINHAMENTO COMPORTAMENTAL E TÉCNICO

Roberto Júnior Cassimiro Campolina
Operations Research and Complex Systems Lab (ORCS)
Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brasil
robertojuniorme@gmail.com

André Costa Batista
Departamento de Engenharia Elétrica
Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brasil
andre-costa@ufmg.br

RESUMO

O gerenciamento de atividades de desenvolvimento em projetos de software pode ser um desafio tático complexo, onde a alocação de tarefas a recursos humanos pode se tornar um gargalo. A alocação de tarefas em projetos de software é tradicionalmente modelada considerando apenas restrições técnicas. Este artigo propõe um modelo de Programação Linear Inteira Mista (MILP) para o Problema de Escalonamento de Projetos de Software (SPSP) que incorpora a compatibilidade comportamental. A formulação penaliza o desalinhamento através de uma função polinomial cúbica dos perfis Solides, aliada a um bônus técnico. A resolução de 1380 instâncias no Gurobi Optimizer demonstra que o ruído cognitivo eleva o *makespan* e valida matematicamente a Lei de Brooks, fornecendo uma métrica relevante para o planejamento tático do projeto.

PALAVRAS CHAVE. Pesquisa Operacional, Programação Inteira Mista, Engenharia de Software, Gestão Ágil, SPSP.

Tópicos: Pesquisa Operacional na Indústria, Gestão de Projetos, Otimização Combinatória.

ABSTRACT

The management of development activities in software projects can be a complex tactical challenge, where the allocation of tasks to human resources can become a bottleneck. Task allocation in software projects is traditionally modeled considering only technical constraints. This paper proposes a Mixed-Integer Linear Programming (MILP) model for the Software Project Scheduling Problem (SPSP) that incorporates behavioral compatibility. The formulation penalizes misalignment through a cubic polynomial function of Solides profiles, allied with a technical bonus. Solving 1380 instances in the Gurobi Optimizer demonstrates that cognitive noise increases the makespan and mathematically validates Brooks's Law, providing a relevant metric for the project's tactical planning.

KEYWORDS. Operations Research. Mixed-Integer Programming. Software Engineering. Agile Management. SPSP.

Paper topics: OR in Industry. Project Management. Combinatorial Optimization.

1. Introdução

A indústria de desenvolvimento de software consolidou-se como um dos principais pilares da economia global moderna, caracterizada por ciclos de entrega rápidos e requisitos dinâmicos. Diferentemente de ambientes tradicionais de manufatura, onde o processamento de máquinas é determinístico, o ciclo de vida do software é fundamentalmente dependente do capital humano. Gerenciar esses recursos de forma eficiente é um desafio em fábricas de software, onde a alocação de profissionais e as dependências arquiteturais do código afetam o cronograma do projeto [Dikert et al., 2016; Faria et al., 2022].

Na literatura de Pesquisa Operacional, esse problema é formalizado como o Problema de Escalonamento de Projetos de Software (SPSP), uma variante especializada do clássico *Resource-Constrained Project Scheduling Problem* (RCPSP). Dada a natureza combinatória das atribuições de tarefas sob restrições de precedência e capacidade, o SPSP é classificado como fortemente NP-Difícil [Blazewicz et al., 1983; Kolisch e Padman, 1999]. Uma revisão abrangente sobre o SPSP, detalhando as diferentes abordagens e métodos de resolução explorados na literatura, pode ser encontrada no levantamento de Vega-Velázquez et al. [2018].

Modelos matemáticos para o SPSP frequentemente diferenciam as equipes estritamente por suas habilidades técnicas [Alba e Chicano, 2007; Chicano et al., 2011; Pritsker et al., 1969]. Essa simplificação não contabiliza fatores cognitivos e comportamentais inerentes à engenharia de software [DeMarco e Lister, 1987]. Além disso, a não-linearidade da produtividade em projetos de software é documentada na literatura empírica. Brooks [1975] formulou a Lei de Brooks, a qual postula que adicionar força de trabalho a um projeto de software atrasado tende a atrasá-lo ainda mais, devido ao crescimento exponencial no *overhead* de comunicação[1] e à topologia restritiva de precedências. Embora seja um conceito recorrente na gestão ágil, essa dinâmica raramente é estruturada como uma variável endógena em modelos analíticos de MILP [Ryan e Cho, 2013; Acuña et al., 2006].

Este artigo visa representar o alinhamento comportamental por meio de um componente numérico dentro de um modelo de otimização matemática. Utilizando a metodologia psicométrica Solides, que categoriza os profissionais em Executores, Comunicadores, Planejadores e Analistas [Costa e Lima, 2018], o modelo ajusta o tempo nominal de execução de uma tarefa com base na compatibilidade psicológica e no excedente técnico do indivíduo. O estudo integra perfis comportamentais em uma formulação SPSP, avalia uma função de penalidade polinomial cúbica, analisa os efeitos matemáticos da Lei de Brooks e explora o planejamento de capacidade tática por meio de um mapa de calor.

O restante deste artigo está organizado da seguinte forma. A Seção 2 detalha a modelagem matemática e a função de ajuste de tempo. A Seção 3 descreve a metodologia experimental e a geração das instâncias computacionais. A Seção 4 apresenta os resultados obtidos pelo otimizador e as análises gerenciais. Por fim, a Seção 5 conclui o trabalho e propõe diretrizes para pesquisas futuras.

2. Modelagem Matemática

A premissa central deste modelo é que a produtividade técnica é impactada pelo perfil do desenvolvedor. A formulação MILP proposta calcula um tempo de execução ajustado *a priori* para cada par tarefa-desenvolvedor com base em características técnicas e comportamentais combinadas.

[1] Refere-se ao tempo e esforço consumidos na troca de informações e no alinhamento entre os membros da equipe. Conforme o número de integrantes em uma mesma atividade aumenta, a complexidade das linhas de comunicação cresce exponencialmente, gerando perdas progressivas na produtividade líquida.

2.1. Conjuntos, Parâmetros e Variáveis

Conjuntos e Índices

M	Conjunto de desenvolvedores, indexado por $i \in \{1, \ldots, \lvert M \rvert\}$.
N	Conjunto de tarefas no *backlog*, indexado por $j, k \in \{1, \ldots, \lvert N \rvert\}$.
S_{max}	Conjunto de *sprints* disponíveis, indexado por $s \in \{1, \ldots, \lvert S_{max} \rvert\}$.
H	Conjunto de habilidades técnicas, indexado por $h \in \{1, \ldots, \lvert H \rvert\}$.
P	Conjunto dos 4 quadrantes comportamentais, indexado por $p \in \{1, \ldots, 4\}$.

Parâmetros

Ht_{jh}	Nível técnico mínimo h exigido pela tarefa j.
Hd_{ih}	Nível técnico atual do desenvolvedor i na habilidade h.
Pt_{jp}	Fração do perfil comportamental p requerido para a tarefa j.
Pd_{ip}	Fração do perfil comportamental p do desenvolvedor i.
T_j^{base}	Tempo base estimado para concluir a tarefa j (horas).
Cap_{is}	Capacidade disponível (horas) do desenvolvedor i na *sprint* s.
δ_s	Duração de cada *sprint* em dias corridos.
j_{lab}	Horas úteis, que podem ser trabalhadas por dia.
$Pred(j)$	Conjunto de tarefas que precedem estritamente a tarefa j.

Variáveis de Decisão

$x_{ijs} \in \{0, 1\}$	1 se a tarefa j for atribuída ao desenvolvedor i na *sprint* s, 0 caso contrário.
$y_s \in \{0, 1\}$	1 se a *sprint* s for utilizada, 0 caso contrário.
$TF_s \geq 0$	Variável contínua do tempo final acumulado (dias) na *sprint* s.
$T^{max} \geq 0$	Variável contínua representando o *Makespan* do projeto.

2.2. Cálculo Dinâmico do Tempo (T_{ij})

O ajuste numérico do tempo de execução é estruturado em três etapas que quantificam o impacto das variáveis humanas na produtividade:

Etapa 1: Excedente Técnico (ET_{ij})
Avalia a qualificação técnica excedente do recurso atribuído. Se o desenvolvedor superar o requisito mínimo estipulado, a execução da tarefa é acelerada. Um fator escalar de 0.5 é incorporado à equação como coeficiente de amortização, suavisando o ganho de produtividade para ser mais condizente com a realidade prática:

$$ET_{ij} = 0.5 \cdot \left[\frac{1}{|H|} \sum_{h \in H} \left(\max(0, Hd_{ih} - Ht_{jh}) \right)^2 \right] \quad \forall i \in M, \forall j \in N \tag{1}$$

Etapa 2: Erro Comportamental (EC_{ij})
Para quantificar o desgaste cognitivo provocado pelo desalinhamento psicológico, calcula-se

inicialmente a diferença absoluta entre os quadrantes do perfil comportamental requerido pela tarefa e o perfil do desenvolvedor, denotada por x_{ij}. A divisão por 2 atua como um fator de normalização: como os perfis comportamentais são representados por distribuições proporcionais (cuja soma vetorial é sempre 1.0), a maior diferença absoluta possível entre dois perfis diametralmente opostos resulta em 2.0. Essa divisão garante que a métrica x_{ij} convirja estritamente para o intervalo contínuo de $[0, 1]$. Em seguida, aplica-se uma função polinomial de ordem n com um multiplicador de ganho k para penalizar desalinhamentos mais severos. Estes parâmetros foram definidos empiricamente como $n = 3$ e $k = 5.0$, formulação cuja justificativa é detalhada na Seção 4.1:

$$x_{ij} = \frac{1}{2} \sum_{p \in P} |Pd_{ip} - Pt_{jp}| \quad \forall i \in M, \forall j \in N \tag{2}$$

$$EC_{ij} = 5.0 \cdot (x_{ij})^3 \quad \forall i \in M, \forall j \in N \tag{3}$$

Etapa 3: Tempo de Execução Ajustado (T_{ij})
A composição final consolida a interação simultânea das variáveis sobre o tempo nominal, atribuindo o ganho de eficiência propiciado pela qualificação técnica e a penalidade gerada pela inadequação comportamental:

$$T_{ij} = T_j^{base} \times (1.0 + EC_{ij} - ET_{ij}) \quad \forall i \in M, \forall j \in N \tag{4}$$

2.3. Função Objetivo e Restrições

O modelo minimiza o cronograma final de entrega do projeto, representado pelo *makespan*:

$$\min T^{max} \tag{5}$$

A alocação das tarefas está sujeita às restrições operacionais e lógicas descritas a seguir. A primeira restrição garante que cada tarefa seja processada exatamente uma vez e atribuída a um único desenvolvedor no ciclo do projeto:

$$\sum_{i \in M} \sum_{s \in S_{max}} x_{ijs} = 1 \quad \forall j \in N \tag{6}$$

A restrição de qualificação impede a atribuição se o recurso designado não possuir o conhecimento técnico mínimo exigido pela tarefa em todas as habilidades:

$$\sum_{i \in M} \sum_{s \in S_{max}} Hd_{ih} \cdot x_{ijs} \geq Ht_{jh} \quad \forall j \in N, \forall h \in H \tag{7}$$

A jornada de trabalho é limitada pela restrição de capacidade, que restringe o total de horas trabalhadas por um desenvolvedor ao seu limite disponível na respectiva *sprint*:

$$\sum_{j \in N} T_{ij} \cdot x_{ijs} \leq Cap_{is} \quad \forall i \in M, \forall s \in S_{max} \tag{8}$$

Para preservar a topologia do código, a restrição de precedência estrutural força que tarefas sucessoras sejam agendadas em *sprints* estritamente posteriores às de suas predecessoras:

$$\sum_{i \in M} \sum_{s \in S_{max}} s \cdot x_{ijs} \geq \sum_{i \in M} \sum_{s \in S_{max}} s \cdot x_{iks} + 1 \quad \forall j \in N, \forall k \in Pred(j) \tag{9}$$

As lógicas de ativação das *sprints* asseguram que as tarefas só sejam alocadas em ciclos ativos e impedem o surgimento de intervalos inativos (buracos) no planejamento cronológico:

$$\sum_{i \in M} \sum_{j \in N} x_{ijs} \geq y_s \quad \forall s \in S_{max} \tag{10}$$

$$x_{ijs} \leq y_s \quad \forall i \in M, \forall j \in N, \forall s \in S_{max} \tag{11}$$

$$y_{s-1} \geq y_s \quad \forall s \in S_{max} \setminus \{1\} \tag{12}$$

As horas de esforço alocado são convertidas em dias corridos no calendário, estruturando a progressão temporal contínua do trabalho ao longo da *sprint*:

$$\delta_s \cdot (s-1) \cdot y_s + \frac{1}{j_{lab}} \sum_{j \in N} x_{ijs} \cdot T_{ij} \leq TF_s \quad \forall i \in M, \forall s \in S_{max} \tag{13}$$

Por fim, a variável de interesse principal da função objetivo é formalizada, garantindo que o *makespan* englobe o tempo de finalização da última *sprint* utilizada:

$$TF_s \leq T^{max} \quad \forall s \in S_{max} \tag{14}$$

3. Metodologia Experimental

Para garantir a confiabilidade estatística e o isolamento preciso das variáveis, um gerador paramétrico em Python instanciou as matrizes dos parâmetros de entrada de forma pseudoaleatória. O conjunto de dados computacional compreende 1380 instâncias de teste, estruturadas de modo a assegurar 30 amostras independentes por cada cenário analisado. O tempo nominal base (T_j^{base}) foi dimensionado para representar projetos práticos de desenvolvimento de software, induzindo prazos totais na faixa de 30 a 100 dias corridos. As instâncias foram categorizadas em cinco grupos de validação:

- **Escalabilidade do MILP:** Examina o esforço computacional e o crescimento combinatório do modelo de otimização através do dimensionamento progressivo do *backlog* de tarefas ($N \in [10, 80]$).
- **Ruído Comportamental:** Mensura o impacto no prazo sob quatro matrizes de alocação: *Baseline* (sem penalidade), *Natural* (perfis estocásticos), *Crítico* (atrito moderado) e *Extremo* (perfis diametralmente opostos).
- **Lei de Brooks:** Avalia o planejamento tático de capacidade iterando simultaneamente as variáveis de volume de tarefas ($N \in [25, 60]$) e o tamanho da equipe de desenvolvedores alocada no projeto ($M \in [3, 8]$).
- **Densidade de Precedência:** Quantifica os gargalos estruturais e a queda na viabilidade do projeto saturando o acoplamento do grafo de interdependências em até 40% da sua capacidade relacional máxima.
- **Excedente Técnico:** Avalia os ganhos de produtividade de equipes altamente qualificadas e os contrapõe às perdas geradas pelo ruído comportamental, cruzando o aumento da senioridade com o desalinhamento dos perfis para identificar o ponto de compensação entre essas duas variáveis.

As formulações foram processadas através do otimizador Gurobi Optimizer, executado em um ambiente computacional equipado com um processador 13th Gen Intel Core i7-1360P (2.20 GHz) e 16 GB de RAM. A fim de proporcionar um balanceamento equilibrado entre a qualidade da convergência e o tempo de reposta em instâncias com dimensionalidade mais robusta, o limite máximo de tempo global foi definido em 3600 segundos (uma hora), com uma tolerância de otimalidade relativa (MIP Gap) estabelecida em 0,1%.

Para garantir a total reprodutibilidade dos experimentos e a transparência metodológica, o código-fonte completo em Python (incluindo a modelagem MILP, o gerador de instâncias e os scripts de análise visual), bem como os dados brutos de saída, estão disponibilizados publicamente no repositório oficial do projeto, acessível através do link: `https://github.com/rob1913r/behavioral-spsp`.

4. Resultados e Discussão

Esta seção apresenta os resultados computacionais derivados da resolução das instâncias no otimizador. A discussão inicia-se com a avaliação da base teórica da função de penalidade adotada e da eficiência computacional do modelo frente a diferentes volumes de tarefas. Na sequência, os dados são explorados para quantificar o impacto do ruído comportamental no *makespan*, validar matematicamente as restrições de capacidade impostas pela Lei de Brooks e investigar a relação empírica de compensação entre o excedente técnico e o alinhamento psicológico.

4.1. Modelagem Teórica da Penalidade Comportamental

A escolha da função cúbica para o Erro Comportamental baseou-se na análise progressiva do desalinhamento. A Figura 1 divide o domínio do erro $x \in [0, 1]$ nas zonas de Aceitação (0.0 a 0.3), Crítica (0.3 a 0.7) e Extrema (0.7 a 1.0). Notou-se que funções lineares ou quadráticas punem pequenos desvios de forma exagerada, enquanto as quárticas subestimam atritos moderados.

A função cúbica mostrou-se mais realista: ela absorve divergências brandas e impõe penalidades acentuadas apenas ao atingir a faixa crítica. Com o multiplicador empírico $k = 5.0$ (Figura 2), o modelo reflete duas realidades: na zona crítica, o tempo sofre inflações severas (até $3\times$ o valor nominal), representando os gargalos reais de comunicação e retrabalho. Já na zona extrema, a penalidade atinge o limiar de inviabilidade prática ($6\times$), espelhando o ponto de ruptura onde a tarefa excede a capacidade limite da *sprint* e força a rejeição matemática da atribuição.

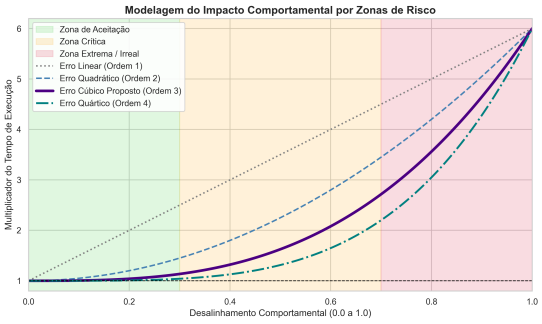

Figura 1: Zonas de impacto por função polinomial.

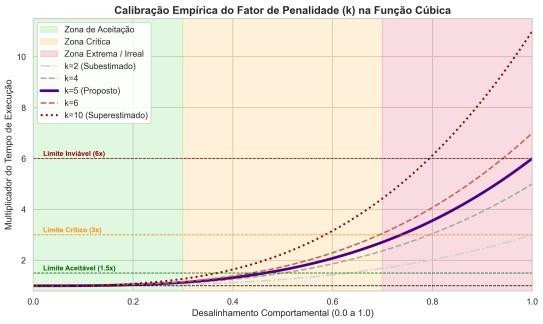

Figura 2: Calibração do multiplicador cúbico k.

4.2. Escalabilidade e Eficiência Computacional do Otimizador

O avanço do esforço computacional e sua aderência às classificações NP-Difícil podem ser observados com clareza em função da escalada do volume de tarefas (Figuras 3 e 4) e do aumento quantitativo das variáveis de decisão ativas no interior do solver (Figuras 5 e 6). Para instâncias contendo até 40 tarefas independentes, a matriz apresentou convergência em menos de 5 segundos de processamento.

Contudo, à medida que a complexidade se aproximou da marca de $N = 80$ tarefas, percebe-se uma transição acentuada na inclinação da curva. O tempo de execução exigido para provar a otimalidade de 0,1% atingiu margens próximas a um minuto nas piores formulações do conjunto, evidenciando o aumento da profundidade da árvore do algoritmo de *Branch and Bound*.

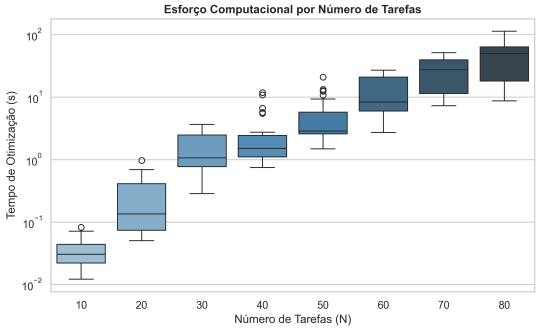

Figura 3: Esforço computacional por Tarefas (N).

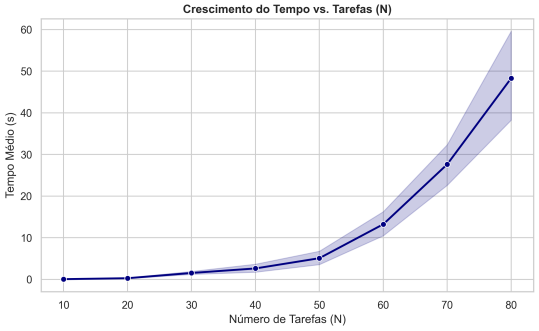

Figura 4: Crescimento do Tempo vs Tarefas.

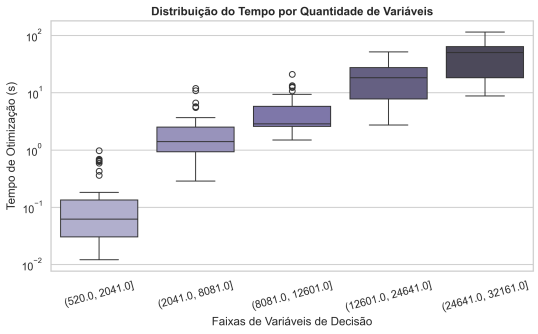

Figura 5: Tempo por quantidade de Variáveis.

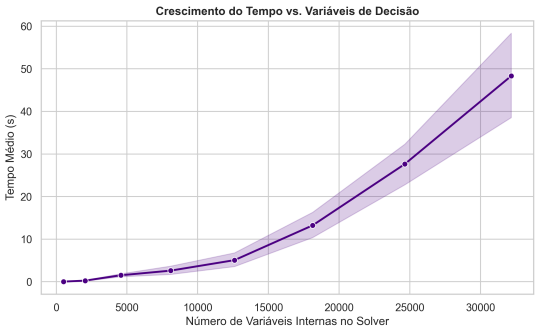

Figura 6: Crescimento do Tempo vs Variáveis.

4.3. Evolução sob o Efeito de Ruído Comportamental

A análise empírica dos quatro cenários de ruído comportamental traz um resultado interessante sobre a dinâmica das equipes e a resposta do modelo matemático. Conforme ilustrado na Figura 7, a transição do cenário *Baseline* (uma equipe ideal e sem atritos) para o cenário *Natural* (perfis estocásticos realistas) resultou em prazos de entrega equivalentes. Esse empate sugere um reflexo direto de como a formulação matemática interage com as decisões do otimizador.

O modelo proposto utiliza uma função de penalidade cúbica, isso significa que, para desvios comportamentais sutis (dentro da Zona de Aceitação, $x_{ij} \leq 0.3$), o impacto gerado

no tempo de execução é matematicamente absorvido. Somado a isso, o algoritmo MILP atua de forma a distribuir as tarefas de maneira a maximizar a afinidade sociotécnica de cada alocação. Na prática, o otimizador consegue contornar os atritos cotidianos, permitindo que uma equipe real produza com a mesma eficiência de um cenário idealizado.

Contudo, existe um limite matemático e prático para essa compensação. Quando o projeto exige alocações mais complexas e o grupo não possui perfis suficientemente compatíveis, o otimizador perde suas rotas de fuga. No cenário *Crítico*, o modelo não consegue mais evitar as penalidades, o que provoca um aumento considerável no *makespan*, passando de uma média de 28 para mais de 40 dias de duração.

Esse efeito de degradação fica ainda mais evidente no cenário *Extremo*, onde as tarefas são obrigatoriamente alocadas a perfis diametralmente opostos. Como mostra a Figura 8, os prazos quase dobram, beirando os 80 dias. Nesse ponto, a incompatibilidade foi tão alta que o tempo de execução ajustado (T_{ij}) chegou a estourar o limite de horas nominais de uma *sprint*. Para que o solver conseguisse encontrar uma solução válida e não declarasse o projeto inviável, foi necessário flexibilizar temporariamente a restrição de capacidade (Cap_{is}). O crescimento do Erro Comportamental (EC) que causou esse impacto na equipe pode ser observado na Figura 9.

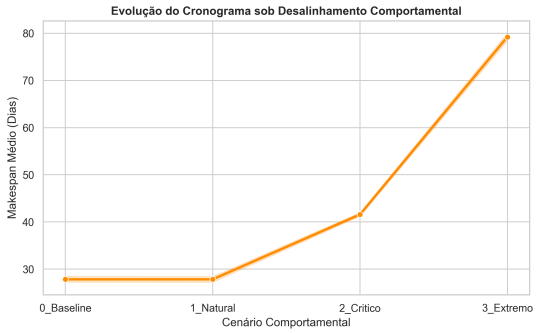

Figura 7: Evolução do cronograma sob ruído.

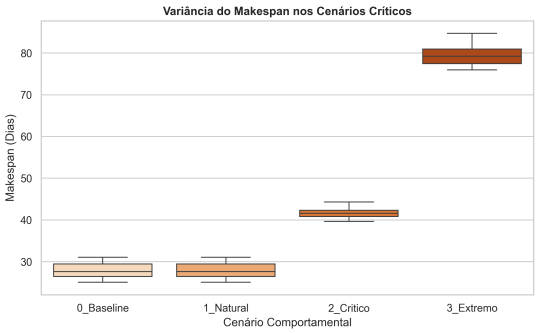

Figura 8: Variância nos cenários comportamentais.

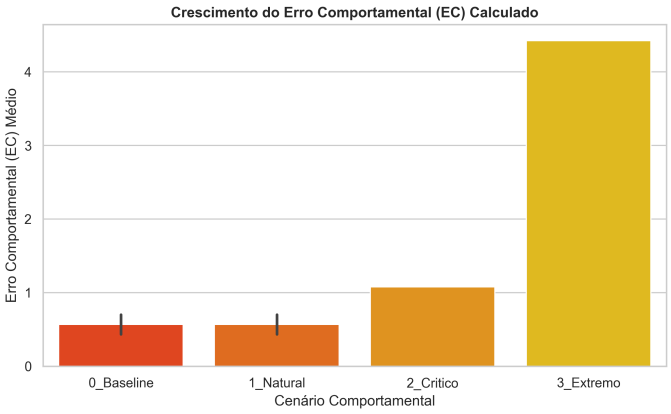

Figura 9: Crescimento do Erro Comportamental (EC) por cenário.

4.4. Planejamento de Capacidade Tática e a Lei de Brooks

Do ponto de vista gerencial, o modelo atua como uma ferramenta direta de suporte à decisão. O mapa de calor tático (Figura 10) permite aos líderes avaliarem rapidamente o custo-benefício de redimensionar a equipe. Fica evidente, por exemplo, que expandir o quadro de 3 para 4 desenvolvedores reduz o *makespan* de forma expressiva. Em contrapartida, saltar de 6 para 8 profissionais traz um ganho baixo no prazo final.

Esse esgotamento da eficiência, isolado na Figura 11 para a demanda de $N = 50$ tarefas, evidencia a Lei de Brooks de forma quantitativa. A lei postula que adicionar força de trabalho a um projeto esbarra em limites de paralelismo e no aumento do custo de comunicação. O algoritmo replicou esse fenômeno empírico: a partir de um determinado tamanho de equipe, a curva de tempo torna-se uma assíntota. Isso indica aos gestores que as dependências do projeto criam um teto onde adicionar mais pessoas deixa de gerar ganhos reais na velocidade de entrega.

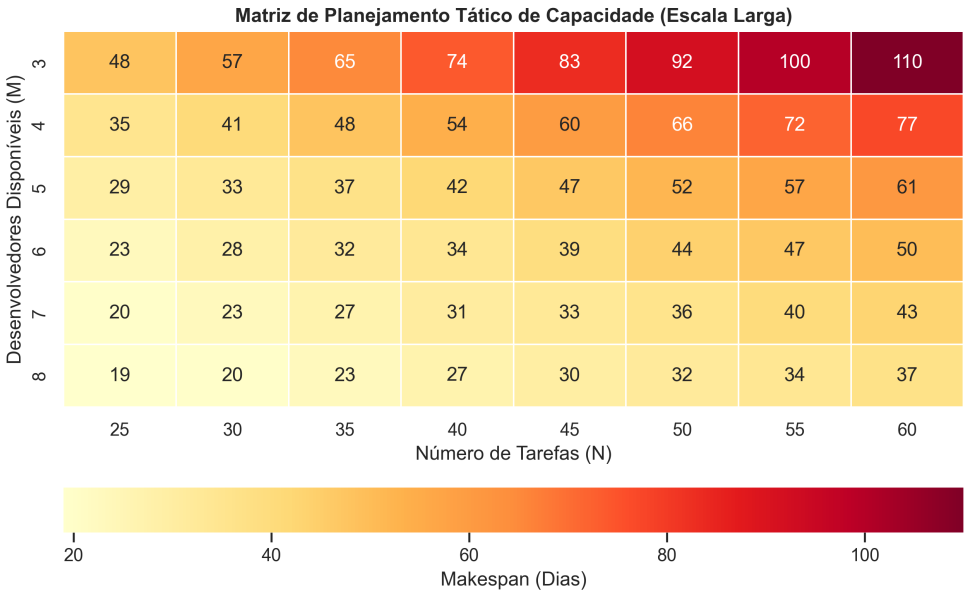

Figura 10: Matriz de Planejamento Tático de Capacidade (Escala Larga).

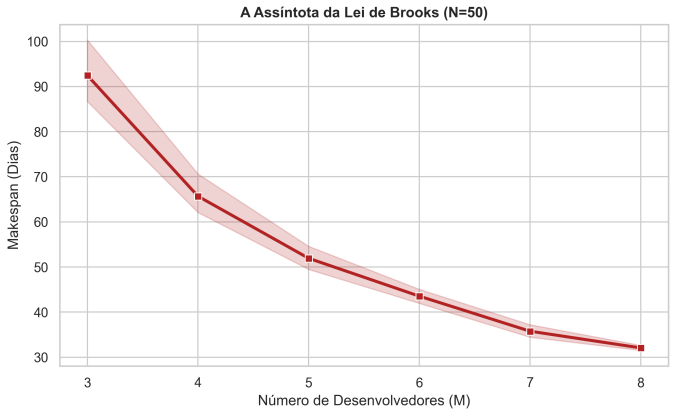

Figura 11: A Assíntota da Lei de Brooks ($N = 50$).

4.5. Limitações de Paralelismo por Densidade de Precedência

A redução nas taxas de eficiência observadas nos times ampliados é uma manifestação direta da tipologia relacional interligando o *backlog* operacional. As restrições ilustradas nos Gráficos 12 e 13 validam a pressuposição de que o acoplamento fechado de um repositório restringe a movimentação otimizada das alocações no solver.

À medida que o fator de amarrações e dependências topológicas excede o limiar dos 30% da capacidade do grafo conectivo, as métricas de tempo inflacionam abruptamente. Mais relevante que a expansão temporal é a taxa de inviabilidade detectada no Gráfico 13; sob tais níveis de acoplamento rígido, os solvers se mostram incapacitados de encaixar o volume das tarefas sem ultrapassar os limites de capacidade em *sprints* totais, reprovando matematicamente o projeto. Estes fatores sugerem a propensão atual das entidades industriais em abandonar matrizes monolíticas de estruturação em favor de microsserviços. Essa abordagem descentralizada quebra a densidade do grafo de dependências, devolvendo ao modelo matemático a flexibilidade combinatória necessária para processar os recursos paralelamente sem invalidar o cronograma.

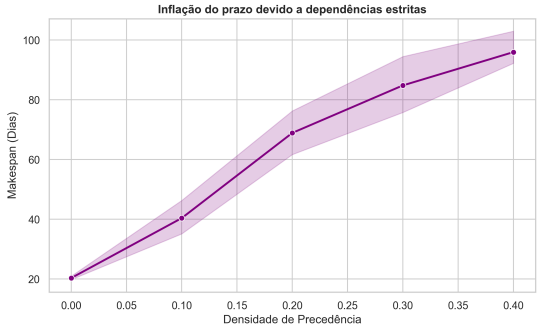

Figura 12: Inflação do prazo devido a precedência.

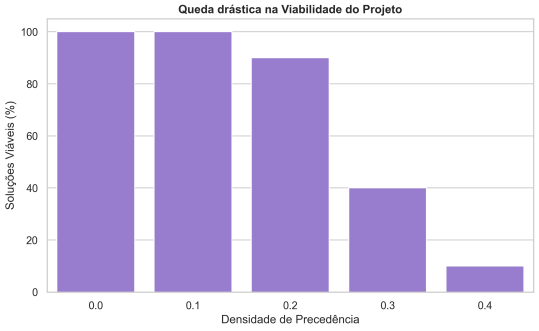

Figura 13: Viabilidade do projeto por densidade.

4.6. Compensação entre Proficiência Técnica e Alinhamento Comportamental

A etapa final da análise explora a relação de troca entre o ganho de produtividade técnica (ET) e as perdas geradas pelo desalinhamento comportamental (EC). A Figura 14 confirma o comportamento tradicional já esperado na literatura: quando mantemos a equipe com um perfil psicológico estável, o aumento da senioridade técnica (maior ET) resulta em reduções consistentes no *makespan*.

No entanto, a principal contribuição desse modelo reside na contraposição dessas duas métricas, revelada na Figura 15. O mapeamento do ponto de interseção das curvas demonstra um resultado importante: um desenvolvedor Júnior ($ET = 0.0$) adequado às exigências comportamentais da tarefa conseguirá executá-la em menos tempo do que um desenvolvedor Sênior ($ET = 1.0$) que esteja operando nas zonas de erro Crítica ou Extrema. Esse cenário sugere que a alta qualificação técnica isolada não é suficiente para compensar os atrasos gerados por uma forte incompatibilidade de perfis na equipe. Consequentemente, práticas de formação de *squads* que incorporem o mapeamento psicológico deixam de ser apenas um diferencial de recursos humanos e passam a significar uma possibilidade real de gerar uma eficiência operacional maior.

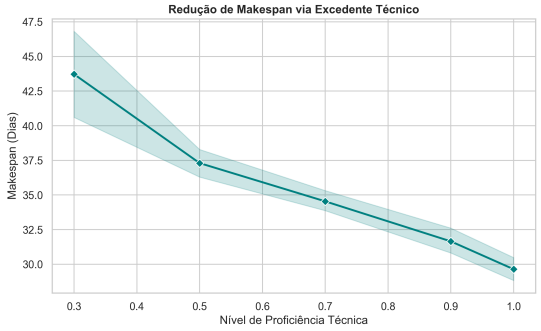

Figura 14: Redução de *Makespan* via Excedente Técnico.

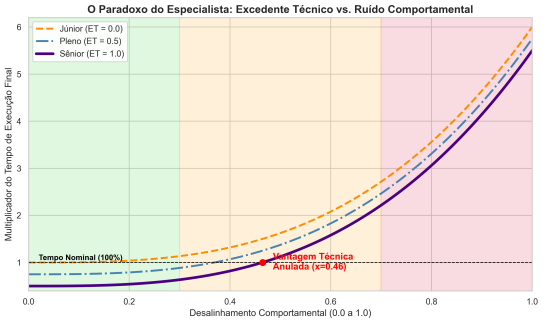

Figura 15: Relação entre Excedente Técnico e Ruído Comportamental.

5. Conclusões e Trabalhos Futuros

Este artigo apresentou uma modelagem matemática para o Problema de Escalonamento de Projetos de Software (SPSP) que busca complementar as abordagens tradicionais ao incorporar não apenas restrições técnicas, mas também variáveis comportamentais. Através de um modelo MILP e da calibração de uma função de penalidade cúbica, os resultados indicam que integrar a compatibilidade psicológica da equipe à alocação de tarefas contribui para a geração de cronogramas mais precisos e realistas.

Os experimentos computacionais geraram resultados quantitativos aplicáveis à gestão tática de projetos. Primeiro, demonstrou-se que o planejamento cego ao perfil humano apresenta desvios significativos, com o cenário de atrito Extremo inflando o *makespan* a níveis que ultrapassam a capacidade estrutural das *sprints*. Segundo, o modelo simulou matematicamente a Lei de Brooks, identificando o ponto de saturação onde o aumento da equipe deixa de acelerar as entregas. Por fim, observou-se uma dinâmica sociotécnica relevante: um desenvolvedor Júnior alinhado ao perfil da tarefa pode finalizá-la em menor tempo do que um Sênior submetido a um alto nível de ruído cognitivo.

Como diretrizes para pesquisas futuras, propõe-se a expansão do modelo matemático para abranger dinâmicas operacionais mais complexas. Entre as melhorias mapeadas, destaca-se a flexibilização topológica para permitir o fracionamento de tarefas extensas ao longo de múltiplas *sprints* e o refinamento das restrições de precedência, viabilizando o início de atividades dependentes dentro de um mesmo ciclo ágil. Além disso, planeja-se introduzir o paralelismo individual (um desenvolvedor atuando em múltiplas tarefas simultâneas) e a alocação de pequenas equipes para uma única atividade. Neste último caso, a modelagem deverá incorporar variáveis para quantificar o *overhead* de comunicação, a curva de adaptação[2] e a sinergia comportamental intra-equipe[3], assegurando que a adição de novos membros complemente a célula de trabalho e não intensifique o atrito cognitivo.

Referências

Acuña, S. T., Gómez, M., e Juristo, N. (2006). How do personality, team processes, and task characteristics relate to job satisfaction and software quality? *Information and Software Technology*, 48(8):622–636.

[2]Refere-se ao tempo e esforço cognitivo iniciais necessários para que a equipe compreenda o contexto, a arquitetura de código e os requisitos específicos de uma tarefa antes de atingir sua produtividade plena.

[3]Métrica teórica que avalia como a combinação de diferentes perfis psicológicos alocados na mesma atividade afeta a colaboração, a dinâmica de resolução de problemas e a eficiência conjunta do grupo.

Alba, E. e Chicano, F. (2007). Software project management with ga's. *Information Sciences*, 177(11):2380–2401.

Blazewicz, J., Lenstra, J. K., e Rinnooy Kan, A. H. (1983). Scheduling subject to resource constraints: classification and complexity. *Discrete Applied Mathematics*, 5(1):11–24.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley Professional.

Chicano, F., Luna, F., Nebro, A. J., e Alba, E. (2011). Using multi-objective metaheuristics to solve the software project scheduling problem. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, p. 1915–1922.

Costa, M. e Lima, F. (2018). Análise de perfil comportamental: A metodologia profiler como ferramenta estratégica. *Revista Brasileira de Recursos Humanos*, 12(4):45–59.

DeMarco, T. e Lister, T. (1987). *Peopleware: Productive Projects and Teams*. Dorset House Publishing Co.

Dikert, K., Paasivaara, M., e Lassenius, C. (2016). Challenges and success factors for large-scale agile transformations: A systematic literature review. *Journal of Systems and Software*, 119:87–108.

Faria, A. G., Machado, G. L., Reis Jr, J., e Silva, L. (2022). Distribuição de tarefas em uma fábrica de software: Equilibrando entregas e lucro em otimização mono-objetivo. *Universidade Federal de Minas Gerais - UFMG*.

Kolisch, R. e Padman, R. (1999). An integrated survey of project scheduling. *Omega - The International Journal of Management Science*, 29(3):249–272.

Pritsker, A. A. B., Watters, L. J., e Wolfe, P. M. (1969). Multiproject scheduling with limited resources: A zero-one programming approach. *Management science*, 16(1):93–108.

Ryan, S. M. e Cho, J. (2013). A mathematical programming approach to the software project scheduling problem. *IEEE Transactions on Engineering Management*, 61(1):145–155.

Vega-Velázquez, M. Á., García-Nájera, A., e Cervantes, H. (2018). A survey on the software project scheduling problem. *International Journal of Production Economics*, 202:145–161.